Encapsulamento

Linguagem de Programação II

Universidade Estadual da Paraíba
Curso: Ciência da Computação

O que vamos aprender hoje?

1. Por que acesso direto aos atributos é perigoso
2. O que é **encapsulamento**
3. O modificador **private**
4. O modificador **public**
5. **Getters**: como ler atributos privados
6. **Setters**: como alterar atributos com validação
7. Boas práticas de encapsulamento
8. Exemplo completo: `Aluno` encapsulada

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

Relembrando o que Sabemos

- Criamos classes com atributos, construtores e métodos;
- No `main`, acessamos diretamente: `a1.nota = 8.5`;
- Isso funciona – mas **tem um problema grave**.

Quem Impede Isso?

```java
Aluno a1 = new Aluno("Maria", 20, 8.5);

// Nada impede que alguem faça isso:
a1.nota  = -99.0;   // nota negativa?
a1.idade = -5;      // idade invalida?
a1.nome  = null;    // sem nome?
```

O Java compila e executa sem reclamar. O objeto fica com dados inválidos.

O Problema Real

Sem nenhuma proteção, **qualquer parte do programa** pode colocar qualquer valor nos atributos – inclusive valores que não fazem sentido para o problema.

Quem Deveria Barrar Isso?

Analogia

Pense num banco. Quando você vai sacar dinheiro, o caixa (ou o sistema) **verifica se você tem saldo** antes de liberar. Não é você quem acessa diretamente o cofre – existe uma **camada de controle** no meio.

Precisamos que a **própria classe** verifique e controle o acesso aos seus dados.

Checkpoint

♡ Checkpoint

- Sem proteção, atributos podem receber valores inválidos de qualquer lugar;
- O compilador não detecta dados errados – só lógica de negócio pode fazer isso;
- Precisamos de um mecanismo para **controlar o acesso** aos atributos.

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

Definição

☺ Definição: Encapsulamento

É o princípio de **esconder os atributos** de uma classe do mundo externo, expondo apenas o que é necessário por meio de **métodos controlados**. Os dados ficam protegidos dentro do objeto – só ele decide como podem ser lidos e alterados.

Uma Analogia

😎 Analogia

Pense no controle remoto da TV. Você aperta os botões – não encosta diretamente nos circuitos internos. O controle é a **interface controlada** que permite usar a TV sem estragar nada por dentro.

Na OO: os **métodos** são os botões. Os **atributos privados** são os circuitos.

O que Muda na Prática

Sem Encapsulamento	Com Encapsulamento
`a1.nota = -99.0;`	`a1.setNota(-99.0);`
Compilador aceita	Setter verifica e rejeita
Dado inválido no objeto	Objeto protegido
Qualquer um altera	Só o setter pode alterar

Checkpoint

♡ Checkpoint

- Encapsulamento = esconder atributos + expor métodos controlados;
- O objeto passa a ser **responsável** pela validade dos próprios dados;
- É implementado com modificadores de acesso (`private`, `public`).

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

O que são Modificadores de Acesso?

☺ Definição: Modificador de Acesso

É uma palavra-chave que define **de onde** um atributo ou método pode ser acessado. Os dois principais que vamos usar agora são `private` e `public`.

private

private

O membro (atributo ou método) só pode ser acessado de dentro da **própria classe**.

- Usado nos **atributos** para protegê-los;
- De fora da classe, é como se não existisse.

Fora da classe

```
a1.nota = 8.5;
```

Erro de compilação!

```
a1.nome = "X";
```

Erro de compilação!

public

public

O membro pode ser acessado de **qualquer lugar**: dentro da classe, de outras classes, do `main`.

- Usado nos **métodos** que queremos expor;
- Construtores quase sempre são `public`.

De qualquer lugar

```
a1.getNota()
```
OK

```
a1.setNota(8.5)
```
OK

Tabela Resumo

Pode acessar de...	`private`	`public`
Dentro da própria classe	✓	✓
De outra classe	×	✓
Do `main`	×	✓
Usado em	atributos	métodos e construtores

Aplicando `private` nos Atributos

```java
public class Aluno {
    private String nome;
    private int    idade;
    private double nota;

    public Aluno(String nome, int idade, double nota) {
        this.nome  = nome;
        this.idade = idade;
        this.nota  = nota;
    }
}
```

Os atributos agora são `private` – de dentro da classe tudo funciona normalmente; de fora, acesso direto é proibido.

O Compilador Agora Protege

```
1 Aluno a1 = new Aluno("Maria", 20, 8.5);
2
3 a1.nota = -99.0;  // ERRO: nota has private access
4 a1.nome = null;   // ERRO: nome has private access
```

Agora o compilador nos avisa. Não é mais possível atribuir valores diretamente de fora.

Erro Comum

☹ Erro Comum

Esquecer de colocar `private` em algum atributo e deixar sem modificador (**default/package**). O atributo fica acessível para classes do mesmo pacote – não é tão protegido quanto `private`. **Regra: sempre escreva `private` nos atributos.**

Checkpoint

♡ Checkpoint

- `private`: só a própria classe acessa – use nos **atributos**;
- `public`: qualquer um acessa – use nos **métodos e construtores**;
- Com `private` nos atributos, acesso direto de fora vira **erro de compilação**.

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

Mas e se Precisar Ler o Atributo?

Se o atributo é `private`, como o `main` consegue **ler** a nota de um aluno?

Criamos um método **público** que retorna o valor – chamado de **getter**.

O que é um Getter?

☺ Definição: Getter

É um método `public` que **retorna** o valor de um atributo privado. Permite leitura controlada – a classe decide o que pode ser lido e como.

- Convenção de nome: `get` + nome do atributo com inicial maiúscula;
- Exemplos: `getNome()`, `getNota()`, `getIdade()`.

Criando Getters

```java
public class Aluno {
    private String nome;
    private int    idade;
    private double nota;

    public String getNome() {
        return nome;
    }

    public int getIdade() {
        return idade;
    }

    public double getNota() {
        return nota;
    }
}
```

Usando os Getters no main

```java
Aluno a1 = new Aluno("Maria", 20, 8.5);

// a1.nota = ...   ERRO - acesso direto proibido
System.out.println(a1.getNota());  // OK: usa getter
System.out.println(a1.getNome());  // OK: usa getter

if (a1.getNota() >= 6.0) {
    System.out.println("Aprovada!");
}
```

Por que Getter e não Acesso Direto?

Acesso direto

`a1.nota`

Expõe o atributo cru. Qualquer um lê e **também escreve**. Sem controle.

Getter

`a1.getNota()`

Leitura controlada. Você pode formatar, calcular ou restringir o que é devolvido. **Escrita não é permitida** por aqui.

😄 Dica

Um getter pode retornar o valor formatado, calculado ou até protegido (ex.: retornar uma cópia de um objeto interno para que o original não seja alterado de fora).

Checkpoint

♡ Checkpoint

- Getter = método `public` que **retorna** um atributo privado;
- Nome padrão: `get` + nome com inicial maiúscula;
- Leitura controlada: a classe decide o que mostrar e como.

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

E se Precisar Alterar o Atributo?

Se o atributo é `private`, como o `main` consegue **atualizar** a nota de um aluno?

Criamos um método **público** que altera o valor – chamado de **setter**.

O que é um Setter?

☺ Definição: Setter

É um método `public` que **recebe** um novo valor e o atribui ao atributo privado. A grande vantagem: podemos **validar** o valor antes de aceitar.

- Convenção de nome: `set` + nome do atributo com inicial maiúscula;
- Exemplos: `setNome(String nome)`, `setNota(double nota)`.

Setter Simples (sem Validação)

```java
public void setNome(String nome) {
    this.nome = nome;
}

public void setIdade(int idade) {
    this.idade = idade;
}

public void setNota(double nota) {
    this.nota = nota;
}
```

Funciona, mas ainda não tem validação. Vamos melhorar isso.

Setter com Validação – O Poder Real

```java
public void setNota(double nota) {
    if (nota < 0.0 || nota > 10.0) {
        System.out.println("Nota invalida: " + nota);
        return; // nao altera o atributo
    }
    this.nota = nota;
}
```

Agora o objeto se protege! Ninguém consegue colocar `-99.0` como nota – o setter rejeita.

Setter com Validação – Mais Exemplos

```java
public void setIdade(int idade) {
    if (idade < 0 || idade > 120) {
        System.out.println("Idade invalida: " + idade);
        return;
    }
    this.idade = idade;
}

public void setNome(String nome) {
    if (nome == null || nome.isEmpty()) {
        System.out.println("Nome nao pode ser vazio.");
        return;
    }
    this.nome = nome;
}
```

Usando os Setters no main

```
Aluno a1 = new Aluno("Maria", 20, 8.5);

a1.setNota(9.5);          // OK: atualiza para 9.5
a1.setNota(-99.0);        // REJEITADO: mensagem de erro
a1.setIdade(-5);          // REJEITADO: idade invalida

System.out.println(a1.getNota()); // imprime 9.5
```

O objeto ficou com 9.5 – o valor inválido -99.0 foi recusado pelo setter.

Getter x Setter – Resumo

	Getter	Setter
Objetivo	Ler um atributo	Alterar um atributo
Retorno	Tipo do atributo	`void`
Parâmetro	Nenhum	Novo valor
Validação	Opcional	**Aqui que vive**
Nome padrão	`getNome()`	`setNome()`

Erro Comum

☹ Erro Comum

Criar um setter que **sempre** aceita qualquer valor, sem nenhuma verificação. Isso é melhor do que acesso direto (ao menos centraliza a atribuição), mas perde o maior benefício do encapsulamento: a validação.

Checkpoint

♡ Checkpoint

- Setter = método `public void` que recebe e **valida** antes de atribuir;
- Nome padrão: `set` + nome com inicial maiúscula;
- A validação no setter **protege o objeto de dados inválidos**.

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

Regras de Ouro do Encapsulamento

1. **Atributos sempre** `private` – nunca deixe público;
2. **Construtores e métodos relevantes:** `public`;
3. **Getters para leitura, setters para escrita** – com validação;
4. **Não crie setter para tudo**: se um atributo não deve mudar depois de criado, não faça setter para ele;
5. **Coloque a lógica de validação dentro do setter**, não no `main`.

Preciso de Getter e Setter para Todo Atributo?

Não!

- Se o atributo não precisa ser lido de fora: **não crie getter**;
- Se o atributo não deve mudar após a criação: **não crie setter**;
- Exemplo: um atributo `dataDeMatricula` provavelmente tem getter mas **não tem setter**.

😄 Dica

Quanto menos você expõe, mais protegida é a classe. Crie getters e setters **só quando necessário**.

Encapsulamento no Construtor

- O construtor também pode fazer validação – afinal, o objeto nasce com aqueles dados;
- Se o valor passado for inválido, você pode: lançar exceção (aprenderemos depois), atribuir um valor padrão seguro, ou imprimir uma mensagem e usar default.

Dica

Uma boa prática: o construtor chama o próprio setter. Assim a validação fica em um só lugar.

Construtor Chamando o Setter

```java
public Aluno(String nome, int idade, double nota) {
    this.nome  = nome;    // pode validar inline
    setIdade(idade);      // reutiliza a validacao do setter
    setNota(nota);        // reutiliza a validacao do setter
}
```

Assim a lógica de validação fica **em um único lugar**: o setter. O construtor só delega.

Checkpoint

♡ Checkpoint

- Sempre `private` nos atributos;
- Crie getter/setter **só quando a necessidade existir**;
- Coloque a validação no setter – não no `main`;
- O construtor pode chamar o setter para reutilizar a validação.

Sumário

1. O Problema sem Encapsulamento
2. O que é Encapsulamento
3. Modificadores de Acesso
4. Getters
5. Setters
6. Boas Práticas
7. Juntando Tudo

A Classe Aluno Encapsulada – Parte 1

```java
public class Aluno {
    private String nome;
    private int    idade;
    private double nota;

    public Aluno(String nome, int idade, double nota) {
        this.nome = nome;
        setIdade(idade);
        setNota(nota);
    }

    // Getters
    public String getNome()  { return nome;  }
    public int    getIdade() { return idade; }
    public double getNota()  { return nota;  }
```

A Classe Aluno Encapsulada – Parte 2

```
    // Setters com validacao
    public void setNome(String nome) {
        if (nome == null || nome.isEmpty()) return;
        this.nome = nome;
    }

    public void setIdade(int idade) {
        if (idade < 0 || idade > 120) return;
        this.idade = idade;
    }

    public void setNota(double nota) {
        if (nota < 0.0 || nota > 10.0) return;
        this.nota = nota;
    }
```

A Classe Aluno Encapsulada – Parte 3

```
1      // Metodos de negocio
2      public void exibirDados() {
3          System.out.println(getNome() + " - "
4              + getIdade() + " anos - nota " + getNota());
5      }
6
7      public boolean calcularSituacao() {
8          return getNota() >= 6.0;
9      }
10 }
```

Note: os métodos de negócio agora usam `getters` internamente – boa prática.

Usando no main

```
Aluno a1 = new Aluno("Maria", 20, 8.5);
Aluno a2 = new Aluno("Joao",  22, 5.0);

System.out.println(a1.getNome()); // Maria
a1.setNota(9.0);                  // atualiza para 9.0
a1.setNota(-5.0);                 // rejeitado silenciosamente
System.out.println(a1.getNota()); // continua 9.0

a2.exibirDados();
if (a2.calcularSituacao()) {
    System.out.println("Aprovado!");
} else {
    System.out.println("Reprovado.");
}
```

Acompanhando Passo a Passo

Linha	O que acontece	a1.nota
`new Aluno("Maria", 20, 8.5)`	construtor chama setNota(8.5)	`8.5`
`a1.setNota(9.0)`	9.0 está no intervalo válido	`9.0`
`a1.setNota(-5.0)`	`-5.0 < 0` – setter rejeita	`9.0`
`a1.getNota()`	retorna o valor protegido	`9.0`

O objeto `a1` nunca teve um valor inválido – a classe se protegeu sozinha.

Diagrama: Antes x Depois do Encapsulamento

Sem Encapsulamento

`main`

↓ acesso direto

`a1.nota = -99`

Objeto com dado inválido

Com Encapsulamento

`main`

↓ chama setter

`a1.setNota(-99)`

↓ setter valida

Objeto protegido

Resumo da Aula

1. Sem encapsulamento, qualquer código pode colocar dados inválidos nos atributos;
2. `private` nos atributos impede acesso direto de fora da classe;
3. `public` nos métodos e construtores permite acesso controlado;
4. **Getter**: método `public` que retorna o atributo (leitura);
5. **Setter**: método `public void` que valida e atribui (escrita);
6. A validação fica no setter – não no `main`;
7. Crie getters e setters **só quando necessário**;
8. O construtor pode chamar os setters para reutilizar a validação.